

**FACULDADE COTEMIG
SISTEMAS DE INFORMAÇÃO**

Arthur Candido, Lucas Albergaria, Mateus Soier

**TRABALHO FINAL IMPLEMENTAÇÃO DE MÁQUINAS ABSTRATAS:
AFD, Autômato de Pilha e Máquina de Turing**

Belo Horizonte
2026

1 INTRODUÇÃO

Este trabalho foi desenvolvido com o objetivo de revisar e implementar as estruturas de Autômatos Finitos Determinísticos (AFD), Autômatos com Pilha (AP) e Máquinas de Turing na linguagem C#. Cada um dos modelos apresentados conta com sua respectiva fundamentação teórica, a arquitetura do autômato desenvolvido e os testes que validam sua execução.

1.1 Fundamentação Teórica

1.1.1 Parte 1

Na parte um, foi solicitada a construção de um AFD (Autômato Finito Determinístico). Podemos definir um AFD como uma 5-tupla $(Q, \Sigma, \delta, q_0, F)$, onde:

- Q é um conjunto finito de estados;
- Σ é um conjunto finito chamado de alfabeto;
- $\delta : Q \times \Sigma \rightarrow Q$ é a função de transição;
- $q_0 \in Q$ é o estado inicial;
- $F \subseteq Q$ é o conjunto de estados de aceitação (ou estados finais).

Com essa estrutura, foi possível construir um AFD capaz de reconhecer as palavras pertencentes a essa linguagem.

$$L_1 = \{ w \in \{a, b\}^* \mid w \text{ termina com } ab \}$$

Figura 1: Lei de formação da Linguagem L1

Fonte: Júlio César da Silva

Entretanto, antes de realizar a construção de um **Autômato Finito Determinístico** devemos saber se essa linguagem é uma linguagem regular, pois os autômatos **AFD** reconhecem linguagens regulares. Então nossa primeira pergunta é:

Questões Reflexivas — Parte 1

1. Por que a linguagem L1 é regular? Apresente a expressão regular equivalente.

Uma linguagem é considerada regular quando é possível construir um autômato finito capaz de reconhecer as palavras pertencentes a ela. Além disso, por definição, toda linguagem regular também pode ser representada e escrita por meio de uma expressão regular equivalente. Esse tipo de linguagem é identificado por possuir padrões de sequências que não exigem contagem, isto é, não é necessário ter memória auxiliar para realizar a computação dos seus elementos.

- Representação de L1 em uma expressão regular equivalente.

Traduzindo a lógica da linguagem, é possível perceber que ela forma palavras compostas pelos símbolos "a" e "b", e que suas cadeias precisam, necessariamente, terminar em "ab". Dessa forma, torna-se viável a construção de uma expressão regular equivalente, uma vez que essa linguagem não exige um gerenciamento de memória para ser computada.

$(a + b)^* ab$

Assim, conseguimos montar a sua expressão regular, onde $(a+b)^*$ representa o conjunto de todas as palavras formadas pelos símbolos a e b, concatenadas com a sequência "ab".

2. Descreva formalmente a 5-tupla do AFD construído, justificando a escolha de cada estado.

$$Q = \{ q_0, q_1, q_2 \}$$

$$\Sigma = \{ a, b \}$$

δ	a	b
q0	q1	q0
q1	q1	q2

q2	q1	q0
----	----	----

Estado inicial = q0

Estado de aceitação = q2

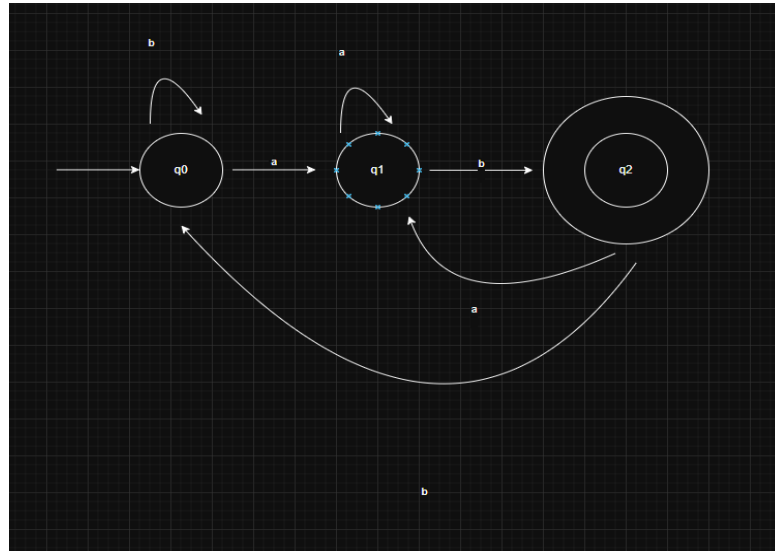


Figura 2: AFD no formato de diagrama

Assim, destacando os estados e suas principais funções:

- **q0 (Estado Inicial):** É o estado inicial do autômato. Ele permanece processando o símbolo "b" enquanto a sequência desejada não se inicia. Sua principal função é capturar o primeiro símbolo "a" da sequência obrigatória "ab" e redirecionar o fluxo para o estado q1.
- **q1:** É o estado responsável por tratar a leitura consecutiva de símbolos "a" (tratando a sequência "aa"). Sua função principal é capturar o símbolo "b" logo após um símbolo "a" (completando a sequência "ab") e realizar a transição para o estado q2.
- **q2 (Estado de Aceitação):** Representa a ocorrência do sufixo "ab". A palavra é aceita se, ao final da leitura, o autômato estiver em q2. Caso novos símbolos sejam lidos, o autômato atualiza seu estado de acordo com a função de transição, sem alterar o critério de aceitação, que depende exclusivamente do estado final após o consumo completo da entrada.

Dessa forma, conseguimos demonstrar o processamento da palavra "aab":

O processo começa no ponto de partida, o estado **q0**. Assim que o autômato lê a primeira letra '**a**', ele muda para o estado **q1** (ou seja, $\delta(q0, a) = q1$).

Quando está em **q1**, se ele ler outra letra '**a**', ele simplesmente continua onde está, no próprio **q1** ($\delta(q1, a) = q1$). Mas, se ele estiver em **q1** e a próxima letra for um '**b**', aí ele avança para o estado **q2** ($\delta(q1, b) = q2$).

Como a palavra termina logo após esse '**b**' e o **q2** é um estado de aceitação (o objetivo final), a palavra é aceita com sucesso!

Dessa forma, conseguimos demonstrar o processamento da palavra “ab”:

O processo começa no ponto de partida, o estado **q0**. Assim que o autômato lê a primeira letra '**a**', ele muda para o estado **q1** (ou seja, $\delta(q0, a) = q1$).

Agora seguindo a transição da próxima letra, se ele estiver em **q1** e ler um '**b**', aí ele avança para o estado **q2** ($\delta(q1, b) = q2$).

Como a palavra termina logo após esse '**b**' e o **q2** é um estado de aceitação (o objetivo final), a palavra é aceita com sucesso!

3. O que aconteceria se a função δ não fosse total? Como você tratou entradas inválidas (símbolos fora do alfabeto)?

No desenvolvimento do simulador de Autômato Finito Determinístico (AFD), utilizou-se um dicionário estruturado para mapear todas as transições possíveis entre os estados do sistema. Durante o processamento de uma fita de entrada, pode ocorrer uma situação em que o símbolo lido não possua uma correspondência mapeada para o estado atual no referido dicionário. Caso essa ausência de transição seja detectada, o fluxo de execução do programa será interrompido imediatamente para evitar comportamentos imprevistos ou loops indefinidos. Como consequência dessa interrupção, o sistema exibirá uma mensagem de erro clara ao usuário, informando explicitamente que a transição desejada não foi previamente definida no escopo do autômato.

Já no caso de entradas inválidas, o código utiliza uma lista que funciona como o alfabeto do AFD, onde ficam guardados todas as letras e símbolos permitidos pelo sistema. Assim que o programa começa a leitura, ele faz uma checagem automática de cada caractere

e, caso o usuário tenha digitado algum símbolo que não pertença a essa lista, o sistema será interrompido imediatamente.

1.1.2 Parte 2

Na parte dois, foi pedido a construção de um Autômato de Pilha(AP) que pode ser representado por uma 6-tupla($M = (Q, \Sigma, \Gamma, \delta, q_0, F)$) onde:

- Q : Conjunto finito de estados.
- Σ : Alfabeto de entrada.
- Γ : Alfabeto da pilha.
- δ : Função de transição (a lógica que dita o próximo estado com base no símbolo lido, no topo atual da pilha e na ação de empilhar/desempilhar).
- q_0 : O estado inicial.
- F : Conjunto de estados de aceitação

O grande diferencial do Autômato de Pilha (AP) é a presença do **alfabeto de pilha**. Como o AP reconhece a classe das **Linguagens Livres de Contexto (LLC)**, ele utiliza essa pilha como uma memória auxiliar de curto prazo. Essa estrutura permite ao autômato realizar contagens e emparelhamentos de símbolos.

$$L_2 = \{ a^n b^n \mid n \geq 1 \}$$

Figura 3: Lei de formação da linguagem L_2 .

Fonte: Júlio César da Silva

Questões Reflexivas — Parte 2

1. Por que L_2 não pode ser reconhecida por um AFD? Demonstre usando o Lema do Bombeamento para linguagens regulares.

Ao analisar a lei de formação dessa linguagem, percebe-se que as palavras que a compõem possuem exatamente a mesma quantidade de símbolos 'a' e 'b'. Diante disso, fica evidente a necessidade de realizar a contagem desses elementos durante o processamento. Para demonstrar formalmente que um Autômato Finito Determinístico (AFD) é incapaz de

reconhecer as palavras geradas por essa linguagem, podemos aplicar o Lema do Bombeamento para linguagens regulares.

De acordo com o Lema do Bombeamento, se uma linguagem for regular, nós podemos “bombear” uma palavra dela, ou seja, repetir uma parte dos símbolos, e a nova palavra gerada ainda precisa respeitar as regras da linguagem.

Aplicando o Lema do Bombeamento, assumimos que a linguagem L_2 é regular e que existe um comprimento de bombeamento p . Escolhemos então a palavra $a^p b^p$, que pertence à linguagem, pois possui a mesma quantidade de símbolos "a" e "b". De acordo com o lema, essa palavra pode ser dividida em três partes x , y e z , sendo que a parte y contém apenas símbolos "a" e pode ser repetida. Ao bombear essa parte, fazendo $i = 2$, obtemos uma nova palavra com mais símbolos "a" do que "b". Dessa forma, a palavra resultante deixa de pertencer à linguagem L_2 , pois não mantém a mesma quantidade de "a" e "b". Isso contradiz o Lema do Bombeamento, provando que L_2 não é uma linguagem regular.

Assim, provando que a Linguagem L_2 não é regular. Logo conseguimos realizar a montagem de um autômato de pilha para realizar o reconhecimento das palavras dessas linguagens .

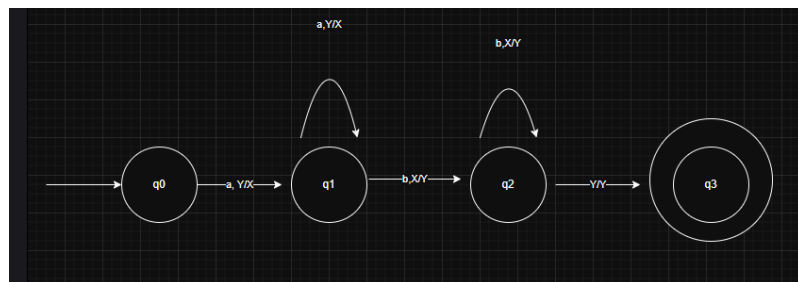


Figura 4: AP no formato de diagrama

- **Observação:** O símbolo lambda por limitações do software virou “Y”.

$$Q = \{q_0, q_1, q_2, q_3\}$$

$$\Sigma = \{a, b\}$$

$$\Gamma = \{X\}$$

$$\delta(q_0, a, \lambda) = (q_1, X)$$

$$\delta(q_1, a, \lambda) = (q_1, X)$$

$$\delta(q_1, b, X) = (q_2, \lambda)$$

$$\delta(q_2, b, X) = (q_2, \lambda)$$

$$\delta(q_2, \lambda, \lambda) = (q_3, \lambda)$$

$$I = q_0$$

$$F = \{q_3\}$$

- **Observação:** Este diagrama do Autômato de Pilha é diferente do que está sendo implementado, pois utiliza 3 estados em vez de 2. Vale ressaltar, ainda, que esse autômato não realiza reconhecimento por pilha vazia, pois não utiliza um símbolo inicial (marcador de fundo) da pilha. Em vez disso, realiza reconhecimento por estado final.

O cálculo manual da palavra “aabb” neste autômato de pilha.

$$(q_0, aabb, \lambda)$$

$$\vdash (q_1, abb, X)$$

$$\vdash (q_1, bb, XX)$$

$$\vdash (q_2, b, X)$$

$$\vdash (q_2, \lambda, \lambda)$$

$$\vdash (q_3, \lambda, \lambda)$$

Como $q_3 \in F$, a cadeia é aceita com sucesso pelo autômato. Vale ressaltar que a fita foi totalmente consumida e a pilha terminou vazia (λ), o que valida o perfeito emparelhamento dos símbolos e garante que a palavra está no estado de aceitação.

O cálculo manual da palavra “aaabbb” neste autômato de pilha.

$$(q_0, aaabbb, \lambda)$$

$$\vdash (q_1, aabbb, X)$$

$$\vdash (q_1, abbb, XX)$$

$\vdash (q1, bbb, XXX)$

$\vdash (q2, bb, XX)$

$\vdash (q2, b, X)$

$\vdash (q2, \lambda, \lambda)$

$\vdash (q3, \lambda, \lambda)$

Como $q3 \in F$, a cadeia é aceita com sucesso pelo autômato. Vale ressaltar que a fita foi totalmente consumida e a pilha terminou vazia (λ), o que valida o perfeito emparelhamento dos símbolos e garante que a palavra está no estado de aceitação.

2. A aceitação por pilha vazia e por estado final são equivalentes em poder de reconhecimento? Demonstre conceitualmente.

Sim, ambos os critérios possuem o mesmo poder de reconhecimento e aceitam exatamente a mesma classe de linguagens (as Linguagens Livres de Contexto). Qualquer Autômato de Pilha (AP) baseado em um modelo pode ser convertido para o outro.

- **De Estado Final para Pilha Vazia:** Adiciona-se um marcador especial no fundo da pilha no início. Quando o autômato original atinge um estado final, o controle é desviado para um novo estado que desempilha recursivamente todos os símbolos até esvaziar a pilha por completo.
- **De Pilha Vazia para Estado Final:** Adiciona-se também um marcador no fundo da pilha. Durante o processamento, se o autômato detectar que sobrou apenas esse marcador (o que significaria pilha vazia no modelo original), ele consome o símbolo e transita para um novo estado definido como estado final de aceitação.

3. Explique o papel do símbolo Z0 na sua implementação.

Na nossa implementação, o símbolo Z0 foi representado pelo caractere 'Z' e atua como o marcador de fundo de pilha (FundoPilha). O seu papel fundamental é indicar formalmente que a pilha está vazia de símbolos da linguagem lida, fornecendo uma condição de parada segura.

Como a exigência do trabalho dita que a aceitação deve ocorrer exclusivamente por pilha vazia (sem verificar estados finais), o autômato realiza todo o processamento de empilhar e desempilhar os elementos da palavra (como os marcadores 'A' e 'B').

Quando a palavra termina de ser consumida e o autômato identifica que o único elemento que restou no topo é o 'Z', ele aciona um último λ -movimento (transição vazia) especificamente desenhado para desempilhar o 'Z'. Ao fazer isso, a pilha atinge o tamanho zero (0 elementos), disparando a condição de aceitação da palavra.

1.1.3 Parte 3 - Máquina de Turing

Na parte três, foi solicitada a construção de uma Máquina de Turing (MT). Podemos definir uma Máquina de Turing Padrão como uma 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{accept}, q_{reject})$, onde:

- Q é um conjunto finito de estados;
- Σ é um conjunto finito chamado de alfabeto de entrada;
- Γ é o alfabeto finito da fita (que contém o alfabeto de entrada e o símbolo de espaço em branco);
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{E, D\}$ é a função de transição parcial (que dita o próximo estado, o símbolo a ser escrito na fita e a direção do movimento do cabeçote: E para a esquerda ou D para a direita);
- $q_0 \in Q$ é o estado inicial;
- $q_{accept} \in Q$ é o estado de aceitação;
- $q_{reject} \in Q$ é o estado de rejeição onde q_{accept} é diferente de q_{reject} ;

Assim conseguimos perceber que entraram duas novas variáveis: o alfabeto da fita e os estados de aceitação e rejeição e suas funções de transições são um pouco diferentes do que vimos. Pois a máquina de Turing reconhece um nível mais elevado das linguagens.

$$L_4 = \{ a^n b^n c^n \mid n \geq 1 \}$$

Figura 5: Lei de formação da linguagem L_4 .

Fonte: Júlio César da Silva

Questões Reflexivas — Parte 3

1. Por que L4 não pode ser reconhecida por um Autômato de Pilha? Qual propriedade da MT possibilita o seu reconhecimento?

Como podemos verificar na lei de formação de L4, as palavras pertencentes a essa linguagem são formadas pelos símbolos "a", "b" e "c", onde suas quantidades devem ser exatamente as mesmas e a ordem de suas ocorrências é rígida (da forma $a^n b^n c^n$). O Autômato de Pilha (AP) consegue realizar o emparelhamento de apenas dois blocos de símbolos por vez (como "a" e "b"), pois utiliza a lógica de empilhar para um símbolo e desempilhar para o outro. Dessa forma, ao consumir o terceiro bloco ("c"), a pilha já estaria vazia, impossibilitando a validação da sua quantidade.

Isso torna a Máquina de Turing (MT) a alternativa necessária para o reconhecimento dessa linguagem. A MT supera essa limitação graças ao seu mecanismo de leitura e escrita na fita, permitindo que o cabeçote se movimente em ambas as direções para marcar os elementos já visitados e garantir que os três símbolos possuam a mesma quantidade.

Assim, tivemos que realizar a construção de uma Máquina de Turing para reconhecer essa linguagem.

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_{\text{accept}}, q_{\text{reject}}\}$$

$$\Sigma = \{a, b, c\}$$

$$\Gamma = \{a, b, c, X, Y, Z, _ \}$$

$$q_0 = \text{estado inicial}$$

$$F = \{q_{\text{accept}}\}$$

ESTADO q_0

$$\delta(q_0, a) = (q_1, X, D)$$

$$\delta(q_0, X) = (q_0, X, D)$$

$$\delta(q_0, Y) = (q_4, Y, D)$$

$$\delta(q_0, b) = (q_{\text{reject}}, b, D)$$

$$\delta(q_0, c) = (q_{\text{reject}}, c, D)$$

$$\delta(q_0, _) = (q_{\text{reject}}, _, D)$$

ESTADO q1

$$\delta(q1, a) = (q1, a, D)$$

$$\delta(q1, Y) = (q1, Y, D)$$

$$\delta(q1, b) = (q2, Y, D)$$

$$\delta(q1, c) = (qreject, c, D)$$

$$\delta(q1, Z) = (qreject, Z, D)$$

$$\delta(q1, _) = (qreject, _, D)$$

ESTADO q2

$$\delta(q2, b) = (q2, b, D)$$

$$\delta(q2, Z) = (q2, Z, D)$$

$$\delta(q2, c) = (q3, Z, E)$$

$$\delta(q2, a) = (qreject, a, D)$$

$$\delta(q2, Y) = (qreject, Y, D)$$

$$\delta(q2, _) = (qreject, _, D)$$

ESTADO q3

$$\delta(q3, a) = (q3, a, E)$$

$$\delta(q3, b) = (q3, b, E)$$

$$\delta(q3, Y) = (q3, Y, E)$$

$$\delta(q3, Z) = (q3, Z, E)$$

$$\delta(q3, X) = (q0, X, D)$$

$$\delta(q3, _) = (qreject, _, D)$$

ESTADO q4

$$\delta(q4, Y) = (q4, Y, D)$$

$$\delta(q4, Z) = (q4, Z, D)$$

$$\delta(q4, _) = (qaccept, _, D)$$

$$\delta(q4, a) = (qreject, a, D)$$

$$\delta(q4, b) = (qreject, b, D)$$

$\delta(q_4, c) = (q_{\text{reject}}, c, D)$

$\delta(q_4, X) = (q_{\text{reject}}, X, D)$

2. Quantos estados foram necessários para a MT de L4? Descreva a estratégia de “marcação” adotada.

Para a construção da Máquina de Turing, foram necessários 7 estados. A estratégia utilizada foi baseada em um sistema de marcação, no qual a máquina relaciona um símbolo "a", um símbolo "b" e um símbolo "c" a cada ciclo de execução.

- q_0 : Procura o primeiro símbolo "a" que ainda não foi processado. Quando encontra um "a", ele é substituído por "X" e a máquina vai para o estado q_1 . Caso encontre um "Y", significa que todos os "a" já foram processados e a máquina segue para q_4 .
- q_1 : Percorre a fita para a direita procurando o primeiro símbolo "b" ainda não marcado. Ao encontrá-lo, ele é substituído por "Y" e a máquina passa para q_2 .
- q_2 : Continua avançando pela fita até encontrar um símbolo "c". Esse símbolo é marcado como "Z" e a máquina segue para q_3 .
- q_3 : Tem a função de retornar ao início da palavra. Para isso, move o cabeçote para a esquerda até encontrar um símbolo "X". Quando isso acontece, o controle volta para q_0 e um novo ciclo é iniciado.
- q_4 : É o estado responsável pela validação final. Nele, a máquina verifica se restaram apenas os símbolos já marcados ("Y" e "Z"). Se encontrar algum símbolo fora do esperado, a palavra é rejeitada.
- q_{accept} : É o estado de aceitação. A máquina chega a esse estado quando todas as verificações foram concluídas com sucesso, indicando que a palavra pertence à linguagem.
- q_{reject} : É o estado de rejeição. A máquina entra nesse estado sempre que encontra alguma situação que não segue as regras da linguagem, como símbolos fora da ordem

correta ou quantidades diferentes de "a", "b" e "c".

Dessa forma, a máquina processa um "a", um "b" e um "c" por vez, marcando-os como "X", "Y" e "Z". O processo se repete até que todos os símbolos tenham sido analisados. Ao final, q4 realiza a última verificação e a máquina segue para qaccept se a palavra for válida ou para qreject caso encontre algum erro.

Dessa forma, é possível demonstrar manualmente o cálculo da palavra 'abc'.

(q0, abc_)

$\vdash (X, q1, bc_)$

$\vdash (XY, q2, c_)$

$\vdash (X, q3, YZ_)$

$\vdash (q3, XYZ_)$

$\vdash (X, q0, YZ_)$

$\vdash (XY, q4, Z_)$

$\vdash (XYZ, q4, _)$

$\vdash (XYZ_, qaccept)$

Como qaccept é o estado de aceitação, a cadeia é aceita com sucesso.

Demonstração manual do cálculo da palavra 'aabbcc'.

(q0, aabbcc_)

$\vdash (X, q1, abbcc_)$

$\vdash (Xa, q1, bbcc_)$

$\vdash (XaY, q2, bcc_)$

$\vdash (XaYb, q2, cc_)$

$\vdash (XaY, q3, bZc_)$

$\vdash (Xa, q3, YbZc_)$

$\vdash (X, q3, aYbZc_)$

$\vdash (q3, XaYbZc_)$

$\vdash (X, q0, aYbZc_)$

$\vdash (XX, q1, YbZc_)$

$\vdash (XXY, q1, bZc_)$ $\vdash (XXYY, q2, Zc_)$ $\vdash (XXYYZ, q2, c_)$ $\vdash (XXYY, q3, ZZ_)$
 $\vdash (XXY, q3, YZZ_)$ $\vdash (XX, q3, YYZZ_)$ $\vdash (X, q3, XYYZZ_)$ $\vdash (XX, q0, YYZZ_)$ $\vdash$
 $(XXY, q4, YZZ_)$ $\vdash (XXYY, q4, ZZ_)$ $\vdash (XXYYZ, q4, Z_)$ $\vdash (XXYYZZ, q4, _)$ $\vdash$
 $(XXYYZZ_ , qaccept, _)$

Como $qaccept$ é o estado de aceitação, a cadeia é aceita com sucesso. Utilizamos essa notação simplificada para poupar espaço e não estender o documento com a demonstração de cada transição.

3. Um computador moderno é mais poderoso que uma Máquina de Turing? Justifique à luz da Tese de Church-Turing.

Não. De acordo com a Tese de Church-Turing, todo algoritmo que pode ser executado por um computador também pode ser computado por uma Máquina de Turing. Embora computadores modernos possuam processadores, memória RAM e sistemas operacionais muito mais eficientes, eles não são mais poderosos do ponto de vista computacional. Isso ocorre porque qualquer operação realizada por um computador pode ser representada por leituras, escritas e mudanças de estado equivalentes às de uma Máquina de Turing. Dessa forma, a Máquina de Turing é considerada o modelo teórico fundamental da computação. Mas podemos destacar que a diferença entre um computador moderno e uma Máquina de Turing está na eficiência e na velocidade de execução, e não na capacidade de computar algoritmos.

2 DESCRIÇÃO DA IMPLEMENTAÇÃO

2.1 AFD (Autômato Finito Determinístico)

Foi criada uma classe para a implementação do AFD com os seguintes atributos:

- **HashSet<string> estados:** Armazena o conjunto de todos os estados que pertencem ao AFD. Ex: q0, q1, q2.
- **HashSet<char> entrada:** Representa o alfabeto do autômato, ou seja, os caracteres válidos que serão aceitos na leitura das palavras.
- **Dictionary<(string estado, char simbolo), string> transicoes:** Dicionário que mapeia a função de transição do autômato. Ele indica qual deve ser o próximo estado com base no estado atual e no símbolo lido.
- **string inicial:** Define o estado inicial por onde o autômato começa a leitura da palavra.
- **HashSet<string> Final:** Armazena o conjunto de estados finais (ou de aceitação) do AFD.

Obs: O **HashSet** é uma coleção própria do C# que armazena elementos únicos. Ele foi utilizado porque, na teoria dos autômatos, os estados e o alfabeto são conjuntos matemáticos (onde não fazem sentido elementos duplicados).

2.2 Funções do Código:

- **public AFD (Construtores):** A classe possui duas formas de ser iniciada. O construtor sem parâmetros define as variáveis com valores fixos de um AFD padrão (que aceita palavras terminadas em "ab"). Já o segundo construtor recebe todas as listas e dicionários por parâmetro, permitindo que as variáveis da classe recebam dados de qualquer outro autômato que seja criado externamente.
- **public bool AceitarPalavra:** Recebe uma string de entrada (input) digitada pelo usuário, definimos a variável estadoAtual como o estado inicial do AFD e fazemos um loop com cada caractere presente na palavra. Dentro deste loop, serão feitos testes de aceitação do input através de implementações if para testar se a palavra possui símbolos aceitos pelo AFD. Caso a palavra possua todos os símbolos permitidos, a palavra irá passar pelo if de dicionários de transições entre os estados. Caso o símbolo não possua uma transição dentro do dicionário, a palavra será rejeitada pelo AFD. Após esta série de testes, caso o estadoAtual termine em um estado de aceitação, a palavra será aceita, senão será negada novamente.

- **public void CarregarPalavrasDeArquivo:** Recebe o caminho de um arquivo de texto e verifica com um if se ele realmente existe no computador. Se existir, o método usa a função ReadAllLines para ler todas as linhas do arquivo de uma vez, salvando cada linha como um item dentro de um array de strings. Em seguida, um loop foreach passa por cada linha, limpa os espaços em branco com a função Trim e faz testes para descobrir se a linha representa uma "palavra vazia" (verificando se a linha está em branco ou se o usuário escreveu as palavras "lambda" ou "λ"). O programa exibe no console o número do teste que está sendo feito e o texto original da linha. Por fim, a palavra já tratada é enviada para a função AceitarPalavra, que roda o autômato e imprime na tela se aquela linha foi aceita ou rejeitada.
- **public AFD Desafio:** Recebe o caminho de um arquivo JSON que contém a estrutura completa de um novo autômato. A função primeiro tenta localizar o arquivo no caminho informado ou na pasta padrão do sistema; caso não encontre, avisa o usuário e cancela o processo. Se o arquivo existir, o código lê todo o seu conteúdo em formato de texto e usa a biblioteca JsonDocument para mapear e navegar pelo arquivo. Através de testes com o método TryGetProperty, a função busca e extrai as propriedades "estados", "alfabeto", "estadoInicial", "estadoFinal" (ou "estadosAceitacao") e o array de "transicoes". Cada dado encontrado é convertido para o seu tipo correto (HashSet, string ou char). O programa processa o array de transições montando as tuplas de (de, letra) e apontando para o estado para. No final, todos esses dados extraídos são usados para criar e retornar uma nova instância da classe AFD totalmente customizada. Se ocorrer qualquer erro de leitura no meio do caminho, o bloco catch captura o erro, exibe uma mensagem e o autômato padrão é mantido.
- **public void ExibirAFD:** Esta função não recebe nenhum parâmetro e serve para mostrar ao usuário como o autômato atual está configurado. Primeiro, ela junta os elementos dos conjuntos de estados e do alfabeto em strings separadas por vírgula usando a função string.Join, imprimindo-as no console. Depois, ela faz um loop foreach para percorrer todo o dicionário de transições. Dentro desse loop, ela separa a chave da transição (que contém o estado de origem e o símbolo) e o valor (que é o estado de destino), exibindo cada linha na tela formatada no padrão matemático da função delta: $\delta(\text{origem}, \text{'símbolo'}) = \text{destino}$. Por fim, o método imprime textualmente qual é o estado inicial e quais são os estados que estão configurados como finais (de aceitação).

2.3 AP (Autômato de Pilha)

Para a implementação do Autômato de Pilha, focamos em não usar bibliotecas de autômatos prontas e construímos a lógica em torno da estrutura Stack do C#, conforme exigido. O código foi dividido em uma classe auxiliar e na classe principal PILHA:

Classe Transição:

- Uma classe simples que atua como nosso dicionário de regras (a função δ). Ela armazena o estado de origem (De), o caractere lido da fita (Le), o caractere que precisa estar no topo da pilha para a transição ocorrer (Desempilha), o estado de destino (Para) e a string contendo o que deve ser empilhado (Empilha).

Classe PILHA:

- FundoPilha e Inicialização: Definimos o caractere 'Z' como marcador de fundo de pilha. O método Reconhecer recebe a palavra, trata o caso de a palavra ser "lambda" (convertendo para uma string vazia) e instancia a pilha inicial já empurrando o 'Z' (Push) para o fundo.
- Simulação e Configuração Instantânea (TentarCaminhos): Este é o coração do simulador. Como implementamos o desafio da linguagem L3 (palíndromos), que é um autômato não-determinístico (ele precisa "adivinhar" o meio da palavra), o simulador foi construído utilizando um algoritmo recursivo de busca. A cada passo processado, o método imprime no console a configuração instantânea atual, mostrando o estado, os elementos concatenados da pilha e o restante da cadeia a ser lida.
- Movimentos-Lambda (λ): No código, os movimentos vazios foram representados pelo caractere reservado '-'. O código verifica se a transição consome uma letra ou se é um movimento vazio. Se for um movimento vazio, a recursão avança sem alterar o índice da letra lida na palavra.
- Empilhamento e Aceitação: Quando uma regra é acionada, o topo da pilha é removido (Pop). Em seguida, caso a regra mande empilhar novos símbolos (ex: empilhar "AZ"), o código percorre a string de trás para frente, empilhando primeiro o 'Z' e depois o 'A',

para que o 'A' fique no topo. Se a pilha atingir a contagem zero ($\text{Count} == 0$) e a palavra tiver sido totalmente lida, a recursão retorna a aceitação. Caso os caminhos testados não esvaziem a pilha ao final, a palavra é rejeitada.

2.4 MT (Máquina de Turing)

Na classe TURING, a fita foi modelada com um `Dictionary<int, char>`, simulando o comportamento de uma fita infinita de forma dinâmica. As regras de transição foram estruturadas em outro dicionário, mapeando o estado e o símbolo atuais para o próximo estado, o caractere de escrita e o movimento do cabeçote.

A execução segue o seguinte fluxo:

O método `Simulacao` inicializa o processo carregando a cadeia na fita, posicionando o cabeçote no início e definindo o estado de partida. No loop principal, a máquina lê o símbolo atual, consulta a tabela de transições, sobrescreve o caractere, move o cabeçote e atualiza o estado.

Para a linguagem $L4 (a^n b^n c^n \mid n \geq 1)$, adotou-se uma estratégia de marcação onde os símbolos "a", "b" e "c" são validados e substituídos por "X", "Y" e "Z". A cada ciclo concluído, o cabeçote retorna ao início da fita.

A validação ocorre no estado `q4`: se a fita contiver apenas marcações e espaços em branco, a palavra é aceita em `qaccept`. Qualquer inconsistência desvia o fluxo para a rejeição em `qreject`. Para segurança, implementou-se uma trava que limita o número máximo de passos, evitando loops infinitos na simulação.

3 RESULTADOS E TESTES

3.1 AFD (Autômato Finito Determinístico)

Para validar o sistema do AFD, foram executados testes no arquivo `entradasAFD.txt`. O programa imprimiu o passo a passo e apresentou o resultado final para cada cadeia, tratando a linha "lambda" como uma linha vazia, obtendo os seguintes resultados:

Teste 1: 'ab'

A palavra 'ab' terminou no estado de aceitação 'q2'. ACEITA!

Teste 2: 'aab'

A palavra 'aab' terminou no estado de aceitação 'q2'. ACEITA!

Teste 3: 'bab'

A palavra 'bab' terminou no estado de aceitação 'q2'. ACEITA!

Teste 4: 'ababab'

A palavra 'ababab' terminou no estado de aceitação 'q2'. ACEITA!

Teste 5: 'ba'

A palavra 'ba' terminou no estado 'q1', que NÃO é de aceitação. REJEITADA!

Teste 6: ' λ ' -> tratando como palavra vazia

A palavra "" terminou no estado 'q0', que NÃO é de aceitação. REJEITADA!

Teste 7: 'b'

A palavra 'b' terminou no estado 'q0', que NÃO é de aceitação. REJEITADA!

3.2 AP (Autômato de Pilha)

Para validar o simulador do AP, foram executados os casos de teste estipulados no arquivo entradas_ap.txt. O sistema imprimiu a configuração instantânea passo a passo e apresentou o resultado final para cada cadeia, obtendo 100% de precisão de acordo com as regras estabelecidas.

Casos de Teste para $L2 = \{a^n b^n \mid n \geq 1\}$:

- ab: ACEITA (Quantidades iguais, $n = 1$, e ordem correta).
- aabb: ACEITA (Quantidades iguais, $n = 2$, e ordem correta).
- aaabbb: ACEITA (Quantidades iguais, $n = 3$, e ordem correta).
- aab: REJEITA (Quantidades distintas: 2 'a', 1 'b'. A pilha não esvazia).
- abb: REJEITA (Quantidades distintas: 1 'a', 2 'b'. Transição trava sem 'a' na pilha).
- ba: REJEITA (Símbolos fora da ordem estipulada pela linguagem).
- λ (vazia): REJEITA (Não cumpre o requisito $n \geq 1$).

- abab: REJEITA (Letras intercaladas, a ordem exigida é todos os 'a' seguidos de todos os 'b').

Casos de Teste para o Desafio L3 (Palíndromos):

- a: ACEITA
- aba: ACEITA
- abba: ACEITA
- ab: REJEITA
- aab: REJEITA

Em todos os testes do desafio L3, a natureza não-determinística da nossa implementação em recursão provou-se eficaz, explorando os movimentos-lambda paralelos e ramificando a execução até que o caminho correto (que esvazia a pilha no meio exato da palavra) fosse encontrado é retornado com sucesso.

3.3 Máquina de Turing

Para validar a Máquina de Turing, foram executados os casos de teste estipulados para a linguagem. O sistema realizou o processamento passo a passo na fita e apresentou o resultado final para cada cadeia, obtendo 100% de precisão de acordo com as regras estabelecidas.

Casos de testes para $L4 = \{a^n b^n c^n \mid n \geq 1\}$:

- **abc**: ACEITA (Quantidades iguais, $n = 1$, e ordem correta).
- **aabbcc**: ACEITA (Quantidades iguais, $n = 2$, e ordem correta).
- **aaabbbccc**: ACEITA (Quantidades iguais, $n = 3$, e ordem correta).
- **aabbc**: REJEITA (Quantidades distintas de símbolos entre os blocos).
- **ab**: REJEITA (Não contém o bloco de símbolos 'c' exigido pela linguagem).
- **abcabc**: REJEITA (Símbolos fora da ordem estipulada pela linguagem).
- **lambda (vazia)**: REJEITA (Não cumpre o requisito $n \geq 1$).

Casos de testes unário

- 1: ACEITA (A máquina adiciona um novo símbolo “1” ao final da fita, produzindo “11”).

- 111: ACEITA (A máquina adiciona um novo símbolo “1” ao final da fita, produzindo “111”).
- 11111: ACEITA (A máquina adiciona um novo símbolo “1” ao final da fita, produzindo “11111”).

4 ANÁLISE COMPARATIVA

Conforme avançamos nos fundamentos teóricos conseguimos perceber que cada autômato ajudou na construção do nível mais elevado que é a máquina de turing. Com isso conseguimos perceber que os AFD reconhecem linguagens regulares e as palavras pertencentes a essa classe, são formadas por símbolos que seguem uma sequência que não precisamos realizar a contagem de elementos.

Conseguindo reconhecer as linguagens de sequências sem realizar contagens nos símbolos entendemos o tipo 3, mas de acordo com a hierarquia de chomsky esse é o nível mais baixo de linguagens. Quando subirmos um pouco o nível, entramos no tipo 2, onde percebemos agora a necessidade de realizar paridade dos símbolos. Assim, graças ao autômato de Pilha conseguimos realizar o reconhecimento de palavras pertencentes a essa linguagem, pois o autômato utiliza-se de uma pilha para conseguir realizar a igualdade com as operações empilha e desempilha e caso a pilha fique vazia, a paridade está correta.

Assim, avançando na Hierarquia de Chomsky, chegamos às Linguagens Sensíveis ao Contexto (Tipo 1) e, por fim, às Linguagens Irrestritas ou Recursivamente Enumeráveis (Tipo 0), que ocupam o nível mais alto dessa classificação. O reconhecimento dessa classe mais poderosa é possível por meio da Máquina de Turing.

Diferentemente do Autômato de Pilha, que utiliza apenas as operações de empilhar e desempilhar para comparar quantidades de dois grupos de símbolos, a Máquina de Turing possui a capacidade de ler, escrever e marcar símbolos diretamente na fita. Graças a essa estratégia de marcação, torna-se possível relacionar e comparar múltiplos conjuntos de símbolos durante o processamento.

Dessa forma, a Máquina de Turing consegue reconhecer linguagens como $L_4 = \{a^n b^n c^n \mid n \geq 1\}$, nas quais é necessário verificar que as quantidades dos símbolos "a", "b" e "c" são exatamente iguais. Isso é possível porque a máquina marca cada ocorrência de "a", "b" e "c" com os símbolos auxiliares "X", "Y" e "Z", permitindo realizar uma correspondência um para um entre os três grupos. Essa capacidade supera as limitações do Autômato de Pilha

convencional, que consegue comparar apenas dois conjuntos de símbolos por meio de operações de empilhamento e desempilhamento

5 CONCLUSÃO

Com este trabalho, tivemos a oportunidade de compreender a fundo as leis de formação de três diferentes sistemas de autômatos, acompanhando de perto a evolução conceitual, teórica e prática de cada um deles. Iniciamos nossa jornada com o Autômato Finito Determinístico (AFD), compreendendo suas limitações práticas devido a uma capacidade de memória estrita, que se restringe apenas ao controle dos seus estados atuais. Em seguida, avançamos na hierarquia da computação ao desenvolver o Autômato com Pilha, modelo que supera essas limitações ao introduzir uma estrutura de memória auxiliar, permitindo o armazenamento, a organização e a manipulação dinâmica de dados conforme os elementos são empilhados ou desempilhados durante a leitura da fita

Por fim, finalizamos nosso aprendizado com o desenvolvimento e a análise da Máquina de Turing, que representa o ápice da computação e o modelo fundamental dos computadores modernos. Graças à sua fita de memória teoricamente infinita e à capacidade de leitura e escrita bidirecional, esse sistema provou ser capaz de simular qualquer algoritmo computável existente, consolidando de forma clara nossa visão sobre a progressão e o poder de processamento dos modelos computacionais.

Ferramentas Utilizadas

Este trabalho utilizou a IDE Visual Studio como ambiente de desenvolvimento. Além disso, recorreu-se ao uso de inteligência artificial para otimizar a redação, corrigir a concordância verbal e sanar dúvidas técnicas sobre a linguagem C#. Por fim, a ferramenta Draw.io foi adotada para a diagramação dos autômatos.

REFERÊNCIAS

CUNHA, Luís Felipe Ignácio. *Aula 12 – Equivalência entre Autômato com Pilha e Gramática Livre do Contexto*. 13 maio 2021. YouTube. Disponível em: <https://www.youtube.com/watch?v=DRWkZx0U76c>. Acesso em: 24 maio 2026.

LINTZMAYER, Carla Negri. *Autômatos com pilha*. 2 mar. 2021. YouTube. Publicado pelo canal Carla Quem Disse. Disponível em: <https://www.youtube.com/watch?v=-rfslM7ouzY>. Acesso em: 24 maio 2026.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elementos de teoria da computação*. 2. ed. Porto Alegre: Bookman, 2002.

MENEZES, Paulo Blauth. *Linguagens formais e autômatos*. 6. ed. Porto Alegre: Bookman, 2010.

RAMOS, Marcus Vinícius Midená; NETO, João José; VEGA, Ítalo Santiago. *Linguagens formais: teoria, modelagem e implementação*. São Paulo: Blucher, 2009.

SIPSER, Michael. *Introdução à teoria da computação*. 2. ed. São Paulo: Cengage Learning, 2007.

SILVA, Júlio Cesar da. *Fundamentos Teóricos da Computação: Slides 5*. Belo Horizonte: Faculdade Cotemig, 2026. Material de aula.

SILVA, Júlio Cesar da. *Trabalho Final: Implementação de Máquinas Abstratas: AFD, Autômato de Pilha e Máquina de Turing*. Belo Horizonte: Faculdade Cotemig, 2026. Material da disciplina Fundamentos Teóricos da Computação.

SILVA, Júlio Cesar da. *Fundamentos Teóricos da Computação: Slides 7*. Belo Horizonte: Faculdade Cotemig, 2026. Material de aula.